19강. 예외처리

- 예외란?
- 예외처리 코드(try ~ catch ~ finally)
- 예외 전달(throws)
- 사용자 정의 예외
- 예외 발생 시키기

19-1. 예외란?

예외와 예외 클래스

- 컴퓨터 하드웨어 문제로 응용프로그램 실행 오류가 발생하는 것을 자바에서 에러(error)라고 한다. 프로그램을 아무리 잘 만들어도 이런 에러에 대처할 방법은 없다.
- 자바에서는 에러 이외에 예외(exception)라고 부르는 오류가 있다. 예외란 잘못된 사용이나 코딩으로 인한 오류를 말한다. 예외가 발생하면 프로그램은 곧바로 종료된다는 관점에서는 에러와 동일하지만, 예외처리를 통해 계속 실행 상태를 유지할 수 있다.

예외 종류

- 일반 예외(Exception)
  - 컴파일러가 예외 처리를 강제함(반드시 예외처리 필요)
  - 주로 외부 자원에 대한 접근이나 입출력(I/O) 작업에서 발생(파일 입출력: FileNotFoundException, 네트워크 통신: IOException 등)
  - 미 처리시 컴파일 오류
- 실행 예외(Runtime Exception)
  - 컴파일러가 예외 처리를 강제하지 않음(선택적 예외처리)
  - 주로 프로그래머의 실수나 논리적 오류에 의해 발생(NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException 등)
  - 미 처리시 실행 중 오류로 멈춤

19-1. 예외란?

- 일반 예외(Exception) 예시

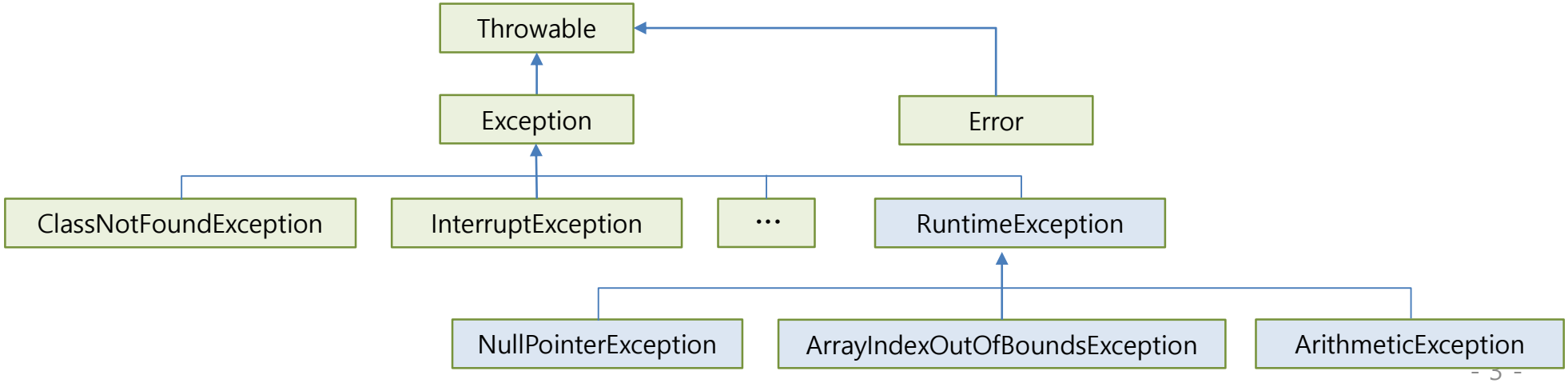
```
try {
    FileInputStream file = new FileInputStream("file.txt");          // 예외 미 처리 시 컴파일 오류
} catch (FileNotFoundException e) {
    System.out.println("파일을 찾을 수 없습니다.");
}
```

- 실행 예외(Runtime Exception) 예시

```
int[] arr = {1, 2, 3};
System.out.println(arr[3]);          // ArrayIndexOutOfBoundsException 발생
```

- 예외의 종류

자바는 예외 발생 시 예외 처리를 위해 예외 클래스로부터 객체를 생성한다. 모든 에러와 예외는 Throwable을 상속받아 만들어지고, 추가적으로 예외 클래스는 java.lang.Exception 클래스를 상속받는다. 자바는 모든 예외 클래스를 표준 라이브러리로 제공한다.



19-2. 예외처리 코드

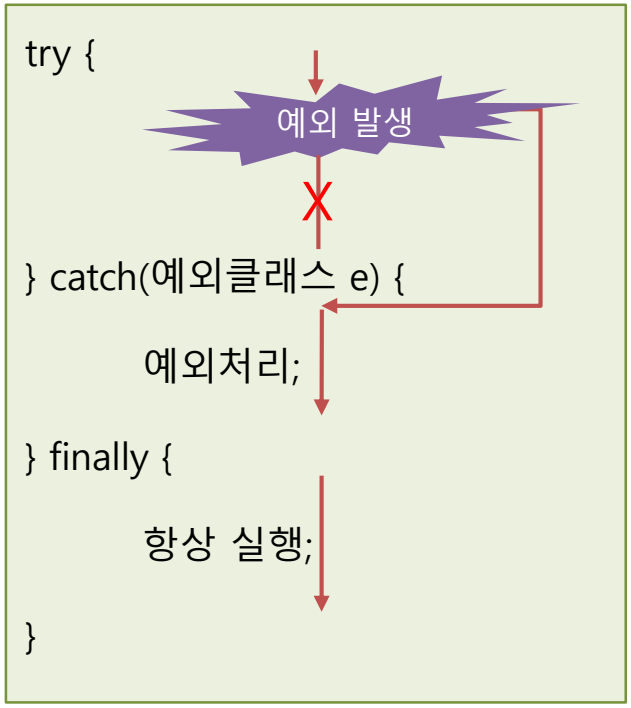
➢ 예외 발생 시 오류로 인한 프로그램 종료를 막고 정상 실행 되도록 처리하는 코드를 예외처리 코드라고 한다. 예외처리 코드는 try ~ catch [~ finally] 블록으로 구성된다.

- 정상 실행

```
try {
    예외 발생 가능 로직 기술

} catch(예외클래스 e) {
        예외처리;
} finally {
        항상 실행;
}
```

- 예외 발생



- try-catch-finally 블록은 생성자와 메서드 내부에서 작성된다.
- finally 블록은 예외 발생 여부와 상관없이 실행되며, 생략 가능하다.
- catch 문은 복수개의 예외처리가 가능하며, Exception을 사용하면 Exception 계열 전부를 캐치 가능

19-2. 예외처리 코드

예외 미 처리(일반 예외)

```
import java.io.FileWriter;

public class ExceptionExam {

    public static void main(String[] args) {
        // FileWrite는 IOException을 발생시킬 가능성 있음
        // try-catch로 감싸지 않으면 컴파일 에러(일반 예외)

        FileWriter fw = new FileWriter("data.txt");
    }
}
```

예외 처리(try-catch 문으로 Exception 계열 예외 발생 준비)

```
public class ExceptionExam {

    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("data.txt");
        } catch (IOException e) {
            // 예외처리를 하여 프로그램이 강제 종료되는 것을 막음
            System.out.println("오류가 발생했습니다.");
        }
    }
}
```

19-2. 예외처리 코드

예외 인스턴스 이용 방법

```
public class ExceptionExam {

    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("data.txt");
        } catch (IOException e) {
            // 상세한 오류 메세지를 얻는 방법
            System.out.println("오류: " + e.getMessage());
            // jvm에서 어떤 경로로 메서드 호출이 일어나서 어디에서
            // 오류가 발생했는지 경로를 출력하는 방법
            e.printStackTrace();
        }
    }
}
```

예외를 Exception으로 처리

```
import java.io.FileWriter;

public class ExceptionExam {

    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("data.txt");
            fw.write("나의 이름은 홍길동");
            fw.close();
        } catch (Exception e) {      // Exception 계열 전부를 캐치 가능
            System.out.println("오류가 발생하였습니다.");
        }
    }
}
```

19-2. 예외처리 코드

잘못된 구문

```
public class ExceptionExam {

    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("data.txt");
            fw.write("나의 이름은 홍길동");
        } catch (IOException e) {
            System.out.println("오류가 발생하였습니다.");
        }
        fw.close();      // try-catch 이후에 close 하면 오류
    }
}
```

try 안에 변수를 설정했기 때문에 fw.close가 작동을 하지 않음

finally 로 해야 하는 처리

```
public class ExceptionExam {

    public static void main(String[] args) {
        FileWriter fw = null;
        try {
            fw = new FileWriter("data.txt");
            fw.write("나의 이름은 홍길동");
        } catch (IOException e) {
            System.out.println("오류가 발생하였습니다.");
        } finally {
            // 파일, DB접속, 네트워크 접속 등에서는
            // 반드시 finally 로 처리해야 함
            try {
                fw.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

19-2. 예외처리 코드

복수 catch 블록을 통한 예외처리

```
public class ExceptionExam {

    public static void main(String[] args) {
        String[] strArr = {"100", "2oo"};

        for(int i = 0; i <= strArr.length; i++) {
            try {
                int iValue = Integer.parseInt(strArr[i]);
                System.out.println("array[" + i + "]: " + iValue);
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("배열 인덱스 초과: " + e.getMessage());
            } catch (NumberFormatException e) {
                System.out.println("숫자로 변환할 수 없음: " + e.getMessage());
            }
        }
    }
}
```

자동 close() 처리되는 try-catch

```
// try 다음 괄호 안에서 생성한 파일 객체는
// finally 블록에서의 close() 생략 가능
try (FileWriter fw = new FileWriter("data.txt")){
    fw.write("나의 이름은 홍길동");
} catch (IOException e) {
    System.out.println("오류가 발생하였습니다.");
}
```

19-3. 예외 전달

예외 발생 시 예외를 처리하는 것이 기본이지만 메서드를 호출한 곳으로 예외를 throws 키워드를 사용하여 떠넘길 수도 있다. throws는 메서드 선언부 끝에 작성하고, 떠넘길 클래스를 쉼표로 구분해서 나열하면 된다.

- 형식

```
리턴 타입 메서드명( 매개변수, ...) throws 예외클래스1, 예외클래스2, ... {
}
```

예외 전달

```
public class ExceptionThrowsExam {
    public static void main(String[] args) {
        try {
            findClass();
        } catch (ClassNotFoundException e) {
            System.out.println("예외 처리: " + e.getMessage());
        }
    }

    public static void findClass() throws ClassNotFoundException {
        // try-catch가 없어도 됨
        Class.forName("java.lang.String2");
    }
}
```

호출한 곳에서 예외처리

19-4. 사용자 정의 예외

표준 라이브러리에 없는 예외를 직접 예외 클래스로 정의하여 사용하는 것을 말한다. 사용자 정의 예외는 컴파일러가 체크하는 일반 예외와 체크하지 않는 실행 예외로 선언할 수 있고, 일반 예외는 Exception의 자식 클래스로 실행 예외는 RuntimeException의 자식 클래스로 선언한다.

- 형식

```
public class XXXException extends [ Exception | RuntimeException ] {
    public XXXException () {
    }

    public XXXException( String message ) {
        super(message);
    }
}
```

기본 생성자

예외 메시지를 입력 받는 생성자

사용자 정의 예외 클래스는 기본 생성자와 예외 메시지를 입력 받는 생성자를 선언해주며, 입력 받은 예외 메시지는 부모 생성자 매개 값으로 전달하는 데, 이 값은 예외 객체의 공통 메서드인 getMessage() 리턴 값으로 사용된다.

사용자 정의 예제 예시

```
public class InsufficientException extends Exception {
    public InsufficientException () {
    }

    public InsufficientException(String message) {
        super(message);
    }
}
```

일반 예외로 선언

두개의 생성자 선언

19-5. 예외 발생 시키기

자바에서 제공하는 표준 예외 및 사용자 정의 예외를 프로그램 코드 상에서 직접 발생시키려면 throw 키워드와 함께 예외 객체를 제공하면 된다. 예외 메시지를 제공하려면 생성자 매개 변수로 전달하면 된다.

- try-catch 블록 사용

```
void method() {
    try {
        ...
        throw new Exception("예외 메시지");
        ...
    } catch(Exception e) {
        String message = e.getMessage();
    }
}
```

- 예외 전달

```
void method() throws Exception {
    ...
    throw new Exception("예외 메시지");
    ...
}
```

19-5. 예외 발생 시키기

자바 표준 예외 발생 시키기(try-catch)

```
public class Person {
    int age;

    public void setAge(int age) {
        try {
            if(age < 0) {
                throw new IllegalArgumentException("나이: " + age + ", [오류 메세지: 나이는 음수가 될수 없습니다.]");
            }

            this.age = age;     // 정상처리되면 필드에 값을 설정
        } catch (IllegalArgumentException e) {
            System.out.println("오류 발생: " + e.getMessage());
        }
    }
}
```

```
public class ExceptionThrowExam {
    public static void main(String[] args) {
        Person person = new Person();
        person.setAge(-30);          // 오류 값을 설정하여 예외 발생
    }
}
```

19-5. 예외 발생 시키기

사용자 정의 예외 발생 시키기(예외 전달)

```
public class InsufficientException extends Exception {
    public InsufficientException () {
    }

    public InsufficientException(String message) {
        super(message);
    }
}
public class Person {
    int age;

    public void setAge(int age) throws InsufficientException {
        if( age < 0 ) {
            throw new InsufficientException("나이 : " + age + ", 오류 메세지: 나이는 음수가 될 수 없습니다.");
        }
    }
}
public class ExceptionThrowExam {
    public static void main(String[] args) {
        Person person = new Person();

        try {
            person.setAge(-30);
        } catch (InsufficientException e) {
            System.out.println("전달 받은 메세지: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

호출한 곳으로 예외 떠넘김

19. 요점 정리

예외

- 일반 예외(Exception: syntax 오류 등 필히 예외처리)
- 실행 예외(Runtime Exception: runtime 오류 등 선택적 예외처리)
- 예외 처리는 실행 예외를 대처하는 것이다.

예외 종류

- API에는 여러 예외적 상황을 표현하는 예외클래스가 있다.
- 예외 클래스는 Throwable을 상속 받아 Error 계열, Exception 계열, RuntimeException 계열로 나뉜다.
- 예외 클래스를 상속하여 사용자 정의 예외 클래스를 정의 할 수 있다.

예외 처리

- try ~ catch 문은 try 블록 내에 예외발생 시 catch 블록으로 처리가 이관된다.
- finally 블록은 예외 발생 여부와 상관없이 실행되므로 반드시 해야 하는 최종 처리를 할 수 있다.
- Exception 계열 예외가 발생할 가능성이 있을 경우 try ~ catch [~ finally] 문을 사용은 필수다.
- throws를 사용하여 예외 처리를 호출하는 주체에 위임할 수 있다.
- throw를 사용하면 자바에서 제공하는 표준 예외와 사용자 정의 예외를 발생시킬 수 있다.

19. 예제

[예제 1]

➢ 아래의 조건을 만족하는 프로그램을 작성, 실행 후 RuntimeException을 발생시키세요.

1. String 형 변수 s를 선언하고, null 을 대입한다.
2. s.length() 의 내용을 표시하려고 한다.

[예제 2] – (java_19_1_ex1_exception: ExceptionEx1)

➢ 예제 1에서 작성한 코드를 수정하여 try-catch 문을 사용하여 아래와 같이 예외처리를 하세요.

1. "NullPointerException 예외를 catch 하였습니다" 를 표시한다.
2. "—stack trace 시작 –" 를 표시한다.
3. stace trace 내용 표시
4. "—stack trace 끝—" 를 표시한다.

19. 예제

[예제 3] – (java_19_7_ex4_alert)

➢ 아래의 조건을 만족하는 프로그램을 작성하세요.

1. 프로그램 기동 직후에 “프로그램 시작 " 을 표시하고 IOException을 임의로 발생시켜 이상 종료 되도록 하세요.